Lecture 2

Thursday, August 27, 2026 1:03 PM

CS 6515: Graduate Algorithms

Lecture 2: Induction and Divide-and-Conquer

Instructors: Sahil Singla and Gerandy Brito

Fall 2026

1 Announcements

1. HW 1 available on Piazza; due next Thursday on Gradescope.
2. Lecture notes uploaded to Piazza.
3. Office hours starting Monday.

2 Recap and plan

Last time we introduced asymptotic notation and merge sort. Its running time satisfies

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + cn, \qquad T(1) = 1,$$

often written more simply as $T(n) = 2T(n/2) + cn$. A recursion tree suggests $T(n) = \Theta(n \log n)$.

Today we will:

- verify recurrence bounds using induction;
- record a useful form of the Master Theorem;
- design a linear-time algorithm for the maximum-difference problem; and
- find the median in deterministic linear time.

3 Proving a recurrence bound by induction

The general recipe is simple: use a recursion tree, experimentation, or another heuristic to *guess* a bound, and then prove that bound by induction.

We use *strong induction* on the integer input size: when proving a bound for $T(n)$, the induction hypothesis is available for every smaller integer. This removes the need to restrict n to powers of two and handles unequal recursive subproblem sizes naturally.

Theorem (merge-sort recurrence). Suppose $c > 0$, $T(1) > 0$ is a constant, and for every integer $n \geq 2$,

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + cn.$$

Then $T(n) = \mathcal{O}(n \log n)$.

→ $\Theta(n \log n)$

Proof. All logarithms in this proof are base two. We will use strongly induction.

Induction hypothesis: $T(m) \le Km \log_2 m$ for every integer $m \ge 2$.

Base case: Choose K sufficiently large so that the claim holds for $m = 2, 3$ and $K \log_2(3/2) \ge c$.

Induction step: For $n \ge 4$, set $a = \lfloor n/2 \rfloor$ and $b = \lceil n/2 \rceil$. Then $a + b = n$, both a and b are smaller than n, and

$$a, b \le \frac{2n}{3}.$$

The strong induction hypothesis therefore gives

$$\begin{aligned} T(n) &\le K(a \log_2 a + b \log_2 b) + cn \\ &\le K(a+b) \log_2(2n/3) + cn \\ &= Kn \log_2 n + Kn \log_2(2/3) + cn \\ &\le Kn \log_2 n. \end{aligned}$$

Thus $T(n) = \mathcal{O}(n \log n)$. □

Exercise. Prove the matching lower bound $T(n) = \Omega(n \log n)$ by strong induction. One useful observation is that $\lfloor n/2 \rfloor, \lceil n/2 \rceil \ge n/3$ for $n \ge 2$. Together with the theorem, this gives $T(n) = \Theta(n \log n)$.

The next example previews the recurrence arising from deterministic median finding. Unlike merge sort, its two recursive calls have different sizes, so no convenient powers-of-two assumption would help.

Theorem (selection-type recurrence). Suppose $c > 0$, T is bounded on constant-size inputs, and for all sufficiently large integers n,

$$T(n) \le T(\lceil n/5 \rceil) + T(\lceil 7n/10 \rceil) + cn.$$

Then $T(n) = \mathcal{O}(n)$.

Proof. We use strong induction.

Induction hypothesis: $T(m) \le Am$ for every integer $m \le n$.

Base case: Choose $A \ge 20c$ and enlarge it, if needed, so that the claim holds for the finitely many small base cases.

Induction step: For n sufficiently large, both recursive arguments are smaller than n, and

$$\begin{aligned} T(n) &\le A\lceil n/5\rceil + A\big(\lceil 7n/10\rceil\big) + cn \\ &\le \frac{9}{10}An + 2A + cn \\ &\le \frac{19}{20}An + 2A \\ &\le An. \end{aligned}$$

The last inequality holds for large n; all smaller values were included among the base cases. □

3.1 A useful form of the Master Theorem

Consider constants $a \ge 1$, $b > 1$, and $d \ge 0$, and the recurrence

$$T(n) = aT(n/b) + \Theta(n^d), \qquad T(1) = \Theta(1).$$

Suppressing floors, ceilings, and constant-size rounding effects, the solution is

$$T(n) = \begin{cases} \Theta(n^d), & d > \log_b a, \\ \Theta(n^d \log n), & d = \log_b a, \\ \Theta\big(n^{\log_b a}\big), & d < \log_b a. \end{cases}$$

This compares the work n^d performed at the root with the total growth $n^{\log_b a}$ arising from the recursive leaves.

4 Maximum difference

Input. Numbers $A[0], A[1], \ldots, A[n-1]$. (Again assume that these numbers are represented using constant number of bits.)

Goal. Find indices $i \le j$ that maximize

$$A[i] - A[j].$$

In other words, find the largest drop from an earlier element to a later one.

Example. For

$$A = [2, 5, 1, 7, 3, 2, 4, 6],$$

an optimal value is $7 - 2 = 5$, attained at $i = 3$ and $j = 5$.

Induction step: For $n \geq 4$, set $a = \lfloor n/2 \rfloor$ and $b = \lceil n/2 \rceil$. Then $a + b = n$, both a and b are smaller than n, and

$$a, b \leq \frac{2n}{3}.$$

The strong induction hypothesis therefore gives

$$\begin{aligned} T(n) &\leq K(a \log_2 a + b \log_2 b) + cn \\ &\leq K(a+b) \log_2(2n/3) + cn \\ &= Kn \log_2 n + Kn \log_2(2/3) + cn \\ &\leq Kn \log_2 n. \end{aligned}$$

Thus $T(n) = \mathcal{O}(n \log n)$. □

Exercise. Prove the matching lower bound $T(n) = \Omega(n \log n)$ by strong induction. One useful observation is that $\lfloor n/2 \rfloor, \lceil n/2 \rceil \geq n/3$ for $n \geq 2$. Together with the theorem, this gives $T(n) = \Theta(n \log n)$.

The next example previews the recurrence arising from deterministic median finding. Unlike merge sort, its two recursive calls have different sizes, so no convenient powers-of-two assumption would help.

Theorem (selection-type recurrence). Suppose $c > 0$, T is bounded on constant-size inputs, and for all sufficiently large integers n,

$$T(n) \leq T(\lceil n/5 \rceil) + T(\lceil 7n/10 \rceil) + cn.$$

Then $T(n) = \mathcal{O}(n)$.

Proof. We use strong induction.

Induction hypothesis: $T(m) \leq Am$ for every integer $m \leq n$.

Base case: Choose $A \geq 20c$ and enlarge it, if needed, so that the claim holds for the finitely many small base cases.

Induction step: For n sufficiently large, both recursive arguments are smaller than n, and

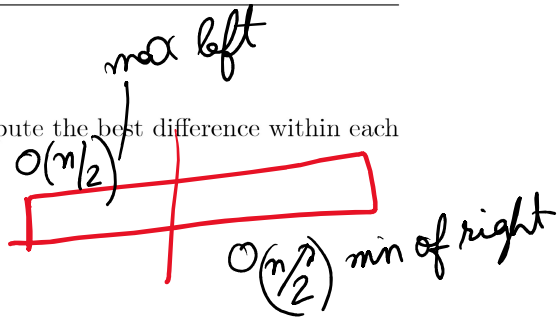
$$\begin{aligned} T(n) &\leq A\lceil n/5 \rceil + A\big(\lceil 7n/10 \rceil\big) + cn \\ &\leq \frac{9}{10} An + 2A + cn \\ &\leq \frac{19}{20} An + 2A \\ &\leq An. \end{aligned}$$

The last inequality holds for large n; all smaller values were included among the base cases. □

4.1 Approach 1: return only the best difference

Split the array into left and right halves and recursively compute the best difference within each half. An optimal pair is one of three types:

1. both indices lie in the left half;
2. both indices lie in the right half; or
3. i lies in the left half and j lies in the right half.



If a recursive call returns only the best difference, computing the crossing solution requires scanning the two halves to find

$$\max(A_{\text{left}}) - \min(A_{\text{right}}).$$

$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$

Thus the recurrence is

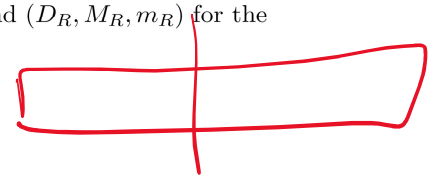
$$T(n) = 2T(n/2) + \mathcal{O}(n) = \Theta(n \log n).$$

4.2 Approach 2: return a richer summary

Each recursive call returns three quantities:

$$D\text{: maximum difference,} \qquad M\text{: maximum element,} \qquad m\text{: minimum element.}$$

For a single element x, return $(0, x, x)$. Given summaries (D_L, M_L, m_L) and (D_R, M_R, m_R) for the two halves, combine them using



$$\begin{aligned} D &= \max\{D_L, D_R, M_L - m_R\}, \\ M &= \max\{M_L, M_R\}, \\ m &= \min\{m_L, m_R\}. \end{aligned}$$

All three quantities take constant time to combine. Therefore

$$T(n) = 2T(n/2) + \mathcal{O}(1) = \Theta(n).$$

Why it works. Every feasible pair lies wholly in the left half, wholly in the right half, or crosses from the left half to the right half. The three candidates in the formula for D are precisely the best solutions of these three types.

Exercise. Prove $T(n) = \Theta(n)$ from the recurrence, both by induction and with a recursion tree. At level r, how much total combine work is performed?

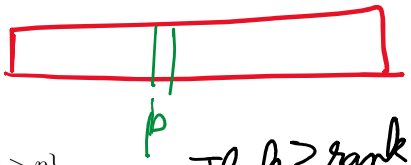
5 Selection and median finding

Input. Distinct numbers $A[0], \ldots, A[n-1]$ and a rank $k \in \{1, 2, \ldots, n\}$.

Goal. Find the k-th smallest element. It has exactly $k-1$ smaller elements and $n-k$ larger elements.

Sorting first takes $\Theta(n \log n)$ time. The selection algorithm of Blum, Floyd, Pratt, Rivest, and Tarjan achieves deterministic linear time by finding a provably good pivot.

Theorem (deterministic selection). The k-th order statistic of n numbers can be found in $\mathcal{O}(n)$ time.



5.1 Proof plan: what makes a pivot good?

Suppose we choose a pivot p and partition the input into

$$L = \{x : x < p\}, \qquad \{p\}, \qquad G = \{x : x > p\}.$$

Once we know $|L|$, we can determine whether the desired element is p, lies in L, or lies in G. We then recurse on only the relevant side.

An arbitrary pivot need not help: if it is the smallest or largest element, the recursive problem can still have size $n-1$. Repeated linear-time partitioning would then lead to the recurrence

$$T(n) \le T(n-1) + cn,$$

which permits quadratic running time. We therefore need a pivot that is bounded away from both extremes; it need not be the exact median.

Definition (good pivot). We call p a *good pivot* if both L and G have size at most

$$\frac{7n}{10} + \mathcal{O}(1).$$

Equivalently, up to constant rounding terms, at least $3n/10$ elements lie on each side of p and can be discarded when we recurse on the other side.

Our proof will follow three steps:

1. In $\mathcal{O}(n)$ nonrecursive work, form about $n/5$ representative elements, then recursively select their median. This costs $T(\lceil n/5 \rceil) + \mathcal{O}(n)$ and produces the pivot p.
2. Prove that this particular p is a good pivot.
3. Partition around p in $\mathcal{O}(n)$ time and recurse on at most $7n/10 + \mathcal{O}(1)$ elements.

Consequently, the running time will satisfy the recurrence previewed in Section 2:

$$T(n) \le T(\lceil n/5 \rceil) + T(\lceil 7n/10 \rceil) + cn = \mathcal{O}(n).$$

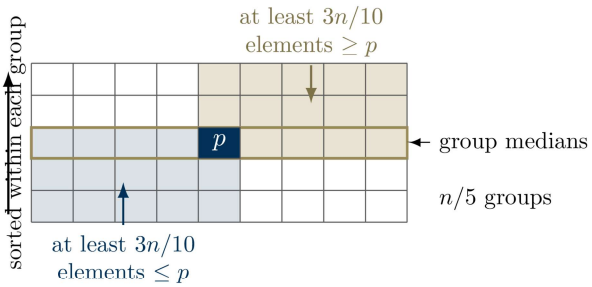
The construction below is designed precisely to establish the missing good-pivot guarantee.

5.2 Finding a good pivot

For clarity, first suppose n is divisible by 5 and ignore harmless rounding terms.

1. Partition the input into $n/5$ groups of five.

2. Sort each group and record its median. Because every group has constant size, this takes $\mathcal{O}(n)$ total time.
3. Recursively find the median p of the $n/5$ group medians.



Columns are shown ordered by their medians only to visualize the counting argument.

Claim. The median-of-medians pivot p has at least $3n/10$ elements no smaller than it and at least $3n/10$ elements no larger than it, up to constant rounding terms.

Proof of the claim. At least half of the group medians are at least p. In each corresponding group, the median and the two larger elements are all at least p. Hence at least

$$3 \cdot \frac{1}{2} \cdot \frac{n}{5} = \frac{3n}{10}$$

elements are at least p. The symmetric argument gives at least $3n/10$ elements at most p. □

5.3 Discarding one side

Partition the input in $\mathcal{O}(n)$ time into

$$L = \{x : x < p\}, \qquad \{p\}, \qquad G = \{x : x > p\}.$$

Then:

- if $k \leq |L|$, recurse on L with rank k;
- if $k = |L| + 1$, return p;
- if $k > |L| + 1$, recurse on G with rank $k - |L| - 1$.

Because p is a good pivot, the recursive side contains at most $7n/10 + \mathcal{O}(1)$ elements.

5.4 Running time

The recursive call that finds the pivot uses an input of size about $n/5$, and the recursive selection call uses an input of size at most $7n/10 + \mathcal{O}(1)$. Grouping, sorting the five-element groups, and partitioning cost $\mathcal{O}(n)$. Thus, ignoring rounding terms,

$$T(n) \leq T(n/5) + T(7n/10) + cn.$$

Exercise. Include the rounding terms and match this recurrence to the selection-type theorem from Section 2. Conclude that deterministic selection runs in $\mathcal{O}(n)$ time.

Takeaway. Divide-and-conquer algorithms become faster when recursive calls return exactly the extra information needed for a cheap combine step, or when a good pivot lets us discard a constant fraction of the input at every step.

2.1 A useful form of the Master Theorem

Consider constants $a \geq 1$, $b > 1$, and $d \geq 0$, and the recurrence

$$T(n) = aT(n/b) + \Theta(n^d), \qquad T(1) = \Theta(1).$$

Suppressing floors, ceilings, and constant-size rounding effects, the solution is

$$T(n) = \begin{cases} \Theta(n^d), & d > \log_b a, \\ \Theta(n^d \log n), & d = \log_b a, \\ \Theta\left(n^{\log_b a}\right), & d < \log_b a. \end{cases}$$

This compares the work n^d performed at the root with the total growth $n^{\log_b a}$ arising from the recursive leaves.

3 Maximum difference

Input. Numbers $A[0], A[1], \ldots, A[n-1]$. (Again assume that these numbers are represented using constant number of bits.)

Goal. Find indices $i \leq j$ that maximize

$$A[i] - A[j].$$

In other words, find the largest drop from an earlier element to a later one.

Example. For

$$A = [2, 5, 1, 7, 3, 2, 4, 6],$$

an optimal value is $7 - 2 = 5$, attained at $i = 3$ and $j = 5$.

3.1 Approach 1: return only the best difference

Split the array into left and right halves and recursively compute the best difference within each half. An optimal pair is one of three types:

1. both indices lie in the left half;
2. both indices lie in the right half; or
3. i lies in the left half and j lies in the right half.

If a recursive call returns only the best difference, computing the crossing solution requires scanning the two halves to find

$$\max(A_{\text{left}}) - \min(A_{\text{right}}).$$

Thus the recurrence is

$$T(n) = 2T(n/2) + \mathcal{O}(n) = \Theta(n \log n).$$

3.2 Approach 2: return a richer summary

Each recursive call returns three quantities:

$$D\text{: maximum difference,} \qquad M\text{: maximum element,} \qquad m\text{: minimum element.}$$

For a single element x, return $(0, x, x)$. Given summaries (D_L, M_L, m_L) and (D_R, M_R, m_R) for the two halves, combine them using

$$\begin{aligned} D &= \max\{D_L, D_R, M_L - m_R\}, \\ M &= \max\{M_L, M_R\}, \\ m &= \min\{m_L, m_R\}. \end{aligned}$$

All three quantities take constant time to combine. Therefore

$$T(n) = 2T(n/2) + \mathcal{O}(1) = \Theta(n).$$

Why it works. Every feasible pair lies wholly in the left half, wholly in the right half, or crosses from the left half to the right half. The three candidates in the formula for D are precisely the best solutions of these three types.

Exercise. Prove $T(n) = \Theta(n)$ from the recurrence, both by induction and with a recursion tree. At level r, how much total combine work is performed?

4 Selection and median finding

Input. Distinct numbers $A[0], \ldots, A[n-1]$ and a rank $k \in \{1, 2, \ldots, n\}$.

Goal. Find the k-th smallest element. It has exactly $k-1$ smaller elements and $n-k$ larger elements.

Sorting first takes $\Theta(n \log n)$ time. The selection algorithm of Blum, Floyd, Pratt, Rivest, and Tarjan achieves deterministic linear time by finding a provably good pivot.

Theorem (deterministic selection). The k-th order statistic of n numbers can be found in $\mathcal{O}(n)$ time.

4.1 Proof plan: what makes a pivot good?

Suppose we choose a pivot p and partition the input into

$$L = \{x : x < p\}, \qquad \{p\}, \qquad G = \{x : x > p\}.$$

Once we know $|L|$, we can determine whether the desired element is p, lies in L, or lies in G. We then recurse on only the relevant side.

An arbitrary pivot need not help: if it is the smallest or largest element, the recursive problem can still have size $n-1$. Repeated linear-time partitioning would then lead to the recurrence

$$T(n) \le T(n-1) + cn,$$

which permits quadratic running time. We therefore need a pivot that is bounded away from both extremes; it need not be the exact median.

Definition (good pivot). We call p a *good pivot* if both L and G have size at most

$$\frac{7n}{10} + \mathcal{O}(1).$$

Equivalently, up to constant rounding terms, at least $3n/10$ elements lie on each side of p and can be discarded when we recurse on the other side.

Our proof will follow three steps:

1. In $\mathcal{O}(n)$ nonrecursive work, form about $n/5$ representative elements, then recursively select their median. This costs $T(\lceil n/5 \rceil) + \mathcal{O}(n)$ and produces the pivot p.
2. Prove that this particular p is a good pivot.
3. Partition around p in $\mathcal{O}(n)$ time and recurse on at most $7n/10 + \mathcal{O}(1)$ elements.

Consequently, the running time will satisfy the recurrence previewed in Section 2:

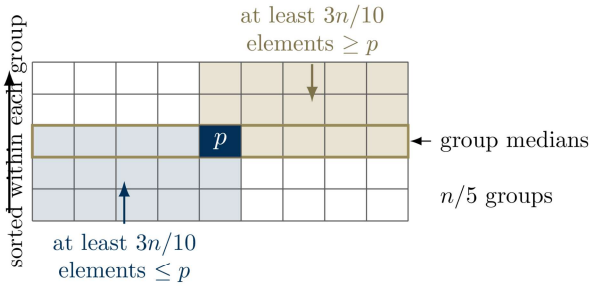
$$T(n) \le T(\lceil n/5 \rceil) + T(\lceil 7n/10 \rceil + 6) + cn = \mathcal{O}(n).$$

The construction below is designed precisely to establish the missing good-pivot guarantee.

4.2 Finding a good pivot

For clarity, first suppose n is divisible by 5 and ignore harmless rounding terms.

1. Partition the input into $n/5$ groups of five.
2. Sort each group and record its median. Because every group has constant size, this takes $\mathcal{O}(n)$ total time.
3. Recursively find the median p of the $n/5$ group medians.



Columns are shown ordered by their medians only to visualize the counting argument.

Claim. The median-of-medians pivot p has at least $3n/10$ elements no smaller than it and at least $3n/10$ elements no larger than it, up to constant rounding terms.

Proof of the claim. At least half of the group medians are at least p. In each corresponding group, the median and the two larger elements are all at least p. Hence at least

$$3 \cdot \frac{1}{2} \cdot \frac{n}{5} = \frac{3n}{10}$$

elements are at least p. The symmetric argument gives at least $3n/10$ elements at most p. □

4.3 Discarding one side

Partition the input in $\mathcal{O}(n)$ time into

$$L = \{x : x < p\}, \qquad \{p\}, \qquad G = \{x : x > p\}.$$

Then:

- if $k \leq |L|$, recurse on L with rank k;
- if $k = |L| + 1$, return p;
- if $k > |L| + 1$, recurse on G with rank $k - |L| - 1$.

Because p is a good pivot, the recursive side contains at most $7n/10 + \mathcal{O}(1)$ elements.

4.4 Running time

The recursive call that finds the pivot uses an input of size about $n/5$, and the recursive selection call uses an input of size at most $7n/10 + \mathcal{O}(1)$. Grouping, sorting the five-element groups, and partitioning cost $\mathcal{O}(n)$. Thus, ignoring rounding terms,

$$T(n) \leq T(n/5) + T(7n/10) + cn.$$

Exercise. Include the rounding terms and match this recurrence to the selection-type theorem from Section 2. Conclude that deterministic selection runs in $\mathcal{O}(n)$ time.

Takeaway. Divide-and-conquer algorithms become faster when recursive calls return exactly the extra information needed for a cheap combine step, or when a good pivot lets us discard a constant fraction of the input at every step.